Orange problem

Operating Systems Unit-4

Name: Hariom N Kini

SRN: PES2UG24CS627

Class: 4K

1

A.

```
gcc -Wall -Wextra -O2 -c test_objects.c -o test_objects.o
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -o test_objects test_objects.o object.o -lcrypto
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

B.

```
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
```

Phase 2

A.

```
make test_tree
./test_tree
rm -f pes test_objects test_tree object.o tree.o index.o commit.o pes.o test_objects.o test_tree.o
rm -rf .pes
gcc -Wall -Wextra -O2 -c test_tree.c -o test_tree.o
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
gcc -Wall -Wextra -O2 -c index.c -o index.o
gcc -o test_tree test_tree.o object.o tree.o index.o -lcrypto
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

B.

```
hnk@Kini:~/PES2UG24CS672-pes-vcs$ xxd .pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
00000000: 626c 6f62 2031 3800 4475 706c 6963 6174  blob 18.Duplicat
00000010: 6520 636f 6e74 656e 740a                 e content.
```

Phase 3:

A.

```
hnk@Kini:~/PES2UG24CS672-pes-vcs$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  pes.h
  untracked:  test_sequence.sh
  untracked:  tree.h
  untracked:  index.h
  untracked:  object.c
  untracked:  commit.h
  untracked:  .gitignore
  untracked:  pes.c
  untracked:  Makefile
  untracked:  test_tree.c
  untracked:  index.c
  untracked:  commit.c
  untracked:  README.md
  untracked:  test_objects.c
  untracked:  tree.c
```

B.

```
hnk@Kini:~/PES2UG24CS672-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776851634 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776851634 6 file2.txt
hnk@Kini:~/PES2UG24CS672-pes-vcs$
```

Phase 4:

A

```
echo "Hello" > hello.txt
./pes add hello.txt
./pes commit -m "Initial commit"

echo "World" >> hello.txt
./pes add hello.txt
./pes commit -m "Add world"

echo "Goodbye" > bye.txt
./pes add bye.txt
./pes commit -m "Add farewell"

./pes log
Initialized empty PES repository in .pes/
[main dfb7aac] Initial commit
Committed: dfb7aaca9373... Initial commit
[main f914bab] Add world
Committed: f914bab654a1... Add world
[main b0a4a02] Add farewell
Committed: b0a4a0201c0a... Add farewell
commit b0a4a0201c0ad7f9b918f46ab1a478fadecd221d1237ebbb75ee6ced4791755f
Author: Hariom_N_Kini<PES2UG24CS672>
Date:   1776852826

    Add farewell

commit f914bab654a1e6db3563c3e0e87078937301124686c3d406181e2d54bd3297e2
Author: Hariom_N_Kini<PES2UG24CS672>
Date:   1776852826

    Add world

commit dfb7aaca937307d16820248ce9ba2f422db8a67f440ece9edfdbf4000a9e7625
Author: Hariom_N_Kini<PES2UG24CS672>
Date:   1776852825

    Initial commit

commit baeaa5bd02e99824baeafeee6e1df4707a431d9a9271dc4bcd745353c80dc16b
Author: Hariom_N_Kini<PES2UG24CS672>
Date:   1776852820
```

B

```
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/a6/882ee4afdc855ea4e39f4e0fc77d90e73a6f72deb383e11934310a10a030f6
.pes/objects/b0/a4a0201c0ad7f9b918f46ab1a478fadecd221d1237ebbb75ee6ced4791755f
.pes/objects/ba/eaa5bd02e99824baeafeee6e1df4707a431d9a9271dc4bcd745353c80dc16b
.pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442
.pes/objects/d8/f28203071e10a3bde605928368bc5b51871287a867d8d10382967099dde814
.pes/objects/df/b7aaca937307d16820248ce9ba2f422db8a67f440ece9edfdbf4000a9e7625
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/objects/f9/14bab654a1e6db3563c3e0e87078937301124686c3d406181e2d54bd3297e2
.pes/refs/heads/main
```

C

```
hnk@Kini:~/PES2UG24CS672-pes-vcs$ cat .pes/refs/heads/main
b0a4a0201c0ad7f9b918f46ab1a478fadecd221d1237ebbb75ee6ced4791755f
hnk@Kini:~/PES2UG24CS672-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
```

Phase 5: Branching and Checkout

Q5.1: How would you implement pes checkout <branch>? What changes and why is it complex?

A branch in PES-VCS is a file inside .pes/refs/heads/ that stores a commit hash. Implementing pes checkout <branch> involves two parts:

1. Update metadata in .pes/

- Modify .pes/HEAD to:

ref: refs/heads/<branch>

- Ensure .pes/refs/heads/<branch> exists.

2. Update the working directory

- Read commit hash from the branch file
- Load the commit object → get root tree
- Recursively:
  - Create directories
  - Write files (blobs)
- Delete files not present in the target tree

Why is this complex?

- Must avoid overwriting user changes

- Must handle:
  - File deletions
  - Directory changes
  - Permissions
- Must keep working directory, index, and commit consistent

Checkout is essentially a **full filesystem reconstruction**, not just switching a pointer.

Q5.2: How to detect "dirty working directory" conflicts?

A dirty working directory means tracked files are modified but not committed.

Steps:

1. For each file in the index:
   - Read file from working directory
   - Compute its hash
   - Compare with index hash
2. If different → file is modified
3. Compare with target branch:
   - Load target commit tree
   - Get hash of same file

Conflict condition:

working_dir_hash ≠ index_hash
AND
target_tree_hash ≠ index_hash

Meaning:

- User modified the file
- Target branch also changed it

Result:

Checkout must **refuse** to prevent data loss.

Q5.3: Detached HEAD — what happens and how to recover?

Normal:

HEAD → branch → commit

Detached HEAD:

HEAD → commit

What happens?

- Commits are created normally
- But no branch points to them
- They become “floating” commits

Problem:

- Can become unreachable
- May be deleted by garbage collection

Recovery:

Option 1:

pes branch new-branch

Option 2:

- Copy commit hash
- Create branch manually pointing to it

Detached HEAD commits are safe temporarily but must be saved with a branch.

Phase 6: Garbage Collection

Q6.1: Algorithm to delete unreachable objects

Goal:

Remove objects not reachable from any branch.

Algorithm (Mark-and-Sweep)

Step 1: Mark phase

- Start from .pes/refs/heads/*
- For each commit:
    - Mark commit
    - Traverse parent
    - Traverse tree
- For each tree:
    - Mark tree

    - Mark all blobs/subtrees

Use a **hash set** to track visited objects.

Step 2: Sweep phase

- Go through .pes/objects/
- Delete objects not in the marked set

Data structure:

Use a hash set for:

- Fast lookup (O(1))
- Avoid duplicate visits

Scale estimation:

- 100,000 commits
- 50 branches

Approx:

- ~100k commits
- ~100k trees
- blobs

Total ≈ 300k to 1M objects

Q6.2: Why is concurrent GC dangerous?

Problem:

GC may delete objects being created by a commit.

Race condition:

1. GC scans reachable objects
2. Commit is writing objects (not yet referenced)
3. GC marks them unreachable
4. GC deletes them
5. Commit finishes → references missing objects

Result:

Repository corruption

How real systems avoid this:

1. Objects are immutable
2. Objects are written before references are updated
3. GC uses locking
4. Recently created objects are protected
5. Deletion is delayed (grace period)

Conclusion:

Garbage collection must be carefully synchronized to avoid deleting valid objects.